

Feature: Services, people and places

From build-plan: feature 12d

Size: heavy - five separate risks with real logic: writing services, a person's or place's lifecycle (turning off, never deleting), who does what (what customers are offered), a work email that changes who gets which email, and a time rule for a service's new length.

Branch: feature/12d-services-people-and-places

Status: verified 2026-10-10: all five steps built and reviewed, then a final review of the whole feature; backend 942, shared 174 and booking component 79 tests pass, the frontend builds and lints.

Goal

The owner manages, on Settings, what customers can book and who does it: each service (its length, buffers, how often start times repeat, whether the customer picks the person, whether it asks the address, live or hidden), the people and places of the business (add, rename, turn off and on again), who does what (the people who do each service and the rooms it needs), and each person's own work email. Changing a service's length keeps every booking and lists the upcoming ones that no longer fit at the new length. After this, nobody edits `booking_link`, `resource`, `booking_link_resource` or a setup file to change any of it.

Design reference

`prototypes/settings.html` : the "What people can book" card (one row per service: name, length, Live or Hidden, Edit; "+ Add a service") and the section list (Hours, Services, Crews, Days off, Calendar, Brand). Crews (feature 19) and the drag handles (no order is stored) are not built. Settings has its section list since the layout fix; Frank is not happy with one long page (Oct 10), so from this feature each section is its own page (decision 1). Shared tokens already ported.

In scope

- Settings becomes one page per section: `/settings/hours` (12a's, moved), `/settings/services`, `/settings/people`; `/settings` opens Hours. The section list links them, the current one marked.
- **Services (12d.1):** the list (name, length, Live or Hidden) and a form to add or change one: name, description (optional), length, buffer before and after, start times every N minutes or every service length, who picks the person (the customer, or the business sends whoever is free), whether it asks the customer's address, live or hidden.
- **People and places (12d.2):** the list (name, person or place, on or off) and a form to add one (name, person or place) or change one (name, on or off). Turning one off never touches a booking: the answer

lists their upcoming bookings, still booked.

- **Who does what (12d.3):** on each service, tick the people who do it and the places it needs; nobody ticked means anyone, no place ticked means no room is needed.
- **Work email (12d.4):** on each person, an optional work email at the business's own domain. When set, the customer's confirmation replies to that person, and the person gets their own new-booking notification.
- **A service's new length (12d.5):** saving a service with a different length lists the upcoming bookings of that service that would no longer fit the hours at the new length.
- Read-only for a role that may not change the business, as 12a.

Out of scope

- Hours (12a, done), closed days and holidays (12e), the booking form and its questions (12f), texts and worker texts (12g), the calendar pick (12h).
- Crews (feature 19), a service's order on the page (no column for it), the booking window's look (`layout` stays `month` , the only one built).
- Deleting a service, person or place (decision 3), changing a person into a place or back (decision 4), moving or cancelling bookings from any list (12b).
- A person's work email on moves, cancellations or reminders: only the new-booking notification and the confirmation's reply (decision 7). Worker texts already cover moves.
- Signing in as a person (crew have no login; the open question for items 13 and 17).

Build loop

Approved one step at a time (AGENTS.md). Heavy: `workflow.stepReview` is `every` , `checkpointCommits` enabled. After a step's plan has Frank's yes, nothing stops until the review: build, tests and checks, tick the box, write the build log entry and commit it to buildlogs, commit and push the step to `feature/12d-services-people-and-places` , `/audit` scoped to the step, then the independent review. The planned stop is after the review. Blocking findings (P0/P1) are fixed by default; P2/P3 are recorded and carried. `/complete` makes the final pass and merges on Frank's yes.

Build steps

- ☒ **12d.1 The owner adds and changes services.** "I open Settings, Services, add 'Cabinet consultation', 45 minutes, the customer picks the person, no address, and it shows in the booking window."
 - Frontend: Settings split into pages (`app/(dashboard)/settings/hours/page.tsx` , `services/page.tsx` , `people/page.tsx` ; `settings/page.tsx` redirects to hours), the section

list in a shared `settings` layout, `components/settings/services-screen.tsx` with the list and one service form (react-hook-form, as 12a), calls in `lib/api-client/settings/`.

- Shared: `serviceValidationSchema` (strict): name 1 to 200, description up to 500 or none, `durationMinutes` 1 to 1440, buffers 0 to 1440, `slotIntervalMinutes` 1 to 1440 or null, `personChoice`, `asksAddress`, `active`. Built from the setup file's service schema where they share a field, so the rules live once.
- Backend: `GET /settings/services` (any member), `POST /settings/services` and `PUT /settings/services/:serviceId (organization: ["update"])`, in `backend/lib/settings/`. A new service gets its slug from `toSlug(name)`, made unique in the business with `-2`, `-3`; renaming never changes it (decision 5). `layout` is set to `month` by the server. Hidden means `active = false`: the booking window no longer lists it, its bookings stay (decision 3). **Done when:** the route tests below pass; and by hand on Summit Painting (dev) at `http://localhost:3400/settings/services`, a new 45-minute service is listed, shows in `/admin/booking-preview/painting-dev` with 45-minute starts, and is gone from there once hidden; `/settings` opens Hours; the read-only screen reads `canEdit` only. *Built 2026-10-10.* Changed while building: the section list shows Hours and Services; People joins it with its page in 12d.2. Reading a Settings page is shared (`settings-section.tsx`, `read-settings-response.ts`), and 12a's `HoursNotice` and `SaveHoursResultType` became `SaveNotice` and `SaveResultType`, since Services uses them too.
- ☒ **12d.2 People and places come and go.** "I add 'Room 2', rename 'Diego (painter)' to 'Diego', turn Tomas off, and he is no longer offered; his Friday booking is still there."
 - Backend: `GET /settings/people`, `POST /settings/people ({ name, kind })` and `PUT /settings/people/:resourceId ({ name, active })`, in `backend/lib/settings/`. A new person follows the business's week (no row: 12a). Never deleted; off is `active = false` (decision 3). The last active person cannot be turned off (409 `last_person`). A name another person or place of the business already has, ignoring case and outer spaces, is refused (409 `name_taken`, decision 6). Turning one off answers with their upcoming confirmed bookings (`upcomingBookings`, 12a's row shape), unchanged.
 - Frontend: `components/settings/people-screen.tsx`, the list and its form; the list of upcoming bookings under the form after turning one off, each linking to its lead. **Done when:** the route tests below pass; and by hand, a place added shows on the list and not on Hours (hours are per person), a person turned off disappears from Hours and is not offered in the booking preview while their booking stays on its lead page, and turning them on again brings both back. *Built 2026-10-10.* Changed while building: `GET /settings/people` also answers the business's `timezone` (the turned-off list shows times in it); a place turned off lists the bookings that use it. Saves of people and places lock the business's row (`for no key update`) so two cannot both find a name free or both find another person on. 12a's `OutsideHoursBookingType` and `OutsideHoursList` became `ListedBookingType` (with `listedBookingOf`) and `ListedBookings` (with a title), shared with this list; this settles F-337. By hand, Summit's services all send whoever is free, so the booking preview never names a person: "not offered" is proved by the route test, not seen in the preview.

- ☒ **12d.3 Who does what.** "On 'Colour consultation' I tick only Marco and Room 1, and the booking window offers only Marco."
 - Backend: `PUT /settings/services/:serviceId/resources ( { peopleIds, placeIds } )` replaces that service's ticks in one transaction. Every id must be a person (in `peopleIds`) or a place (in `placeIds`) of this business, on or off; anything else is 400 with the field. `GET /settings/services` returns each service's `peopleIds` and `placeIds` . The booking side already reads them (`findServiceResources`): nobody ticked means anyone, a turned-off person is never offered.
 - Frontend: on the service form, two lists of ticks (people, places), turned-off ones shown as off; the hints "Nobody ticked: anyone can do it" and "No place ticked: no room needed". **Done when:** the route tests below pass; and by hand, ticking only Marco on a service makes the booking preview offer only Marco for it, and clearing the ticks offers everyone. *Built 2026-10-10*. Changed while building: the service form's one Save sends the service, then its ticks when they changed, by their own call; a new service whose ticks fail to save is changed, never added twice, on the next Save. A service's save and add answers carry its `peopleIds` and `placeIds` . Added: when every one ticked is off, the hint turns red and says nobody is offered (people) or it cannot be booked (places), since the booking side never falls back to anyone then. By hand, Summit's Colour consultation was switched to "the customer picks" so the preview names people.
- ☒ **12d.4 A person's work email.** "I give Pedro pedro@summit-painting.test; a customer booking with Pedro can reply to him, and Pedro gets his own notification."
 - Migration: `resource.workEmail` (text, null), a person's only (check), stored trimmed and lowercased.
 - `PUT /settings/people/:resourceId` takes `workEmail` (an address, or null to clear). It must be at the domain of the business's `senderEmail` (decision 8): another domain is 400 on the field; a business with no sender address set is 409 `no_sending_address` ("Set the business's sending address first").
 - `findBookingEmailRecipients` gains the booked person's work email: the confirmation's reply goes to it, and the person gets the new-booking notification (a new email kind, recorded on the timeline like the others, without the address). Unchanged when unset: no reply-to, no notification to the person (decision 11 of feature 6 still holds for anyone without one). **Done when:** the recipients rule tests and route tests below pass; and by hand with the dev business's Resend test key (or the console sender if none), a booking with a person who has a work email sends the person's notification and sets the confirmation's reply-to, and one with a person without sends exactly what it sent before. *Built 2026-10-10*. Changed while building: the person's email job is queued only when the booked person has a work email at booking time (and reads it again when it runs), so a booking without one queues exactly the two jobs it did; the person is also told of a booking the owner made; a work email that is the business's notification address gets one email, not two; null or an empty field clears it; a place sent one is refused (400). By hand on Summit (sending address at example.com), Pedro got pedro@example.com; two "Hand check" bookings of Exterior estimate stay in `scheduledads_dev`, and Exterior estimate is back to nobody ticked.

- ☒ **12d.5 A service's new length lists the bookings that no longer fit.** "I make the estimate 90 minutes instead of 60, and I am told Maria's Tuesday 4:00 would now run past 5:00; it is still booked as it was."
 - `backend/lib/bookable-hours/apply-length-change-rules.ts` , no database: given the hours, the service's upcoming bookings and the new length, it returns the confirmed bookings still to come that fit as booked and would not fit starting at the same moment with the new length. "Fits" is 12a.2's exported test (`fitsHours`), so clock changes and one-off dates count the same way (decision 9).
 - The service save (12d.1's `PUT`) reads the business's hours, the people's rows and the service's upcoming bookings in its transaction when the length changes, and answers `outsideHours` (12a's row shape); the form shows 12a's list under it. **Done when:** the Simulate page's cases pass as saved tests under the same names; and by hand, a Tuesday 4:00 booking of a 60-minute service with Tuesday ending at 5:00, then the service made 90 minutes, lists that booking, which is unchanged on its lead page. *Built 2026-10-10.* Changed while building: the service save is one transaction that locks the service's row and reads the business's hours shared; a business with no hours lists nothing. `GET /settings/services` also answers the business's `timezone` , for the list's times. Each person's own hours are read by `findPeopleHours` , now shared with the hours save. By hand on Summit, Interior estimate made 90 minutes listed Marta's Wednesday 7:30 with Carlos, still 60 minutes on her lead; put back to 60, which listed nothing.

Files / areas

- `frontend/app/(dashboard)/settings/` (layout, `hours/` , `services/` , `people/`), `frontend/components/settings/` , `frontend/lib/api-client/settings/`
- `packages/shared/zod-validation/` (service, resource, ticks and work-email schemas, exported from the index)
- `packages/shared/db/booking-tables/resource-table.ts` and a new migration (12d.4 only)
- `backend/routes/settings-routes.ts` and its test; `backend/lib/settings/` (find and save services, people, ticks)
- `backend/lib/bookable-hours/apply-outside-hours-rules.ts` (export `fitsHours`), `apply-length-change-rules.ts` and its test (12d.5)
- `backend/lib/email/find-booking-email-recipients.ts` , `send-booking-emails.ts` , `send-and-record-emails.ts` (the new kind) and the notification template (12d.4)
- `backend/lib/errors/refuse.ts` (`last_person` , `name_taken` , `no_sending_address`)

Data / contracts

All under the dashboard CORS, CSRF and no-store middleware and the `booking` module; reads for any member, writes need `requirePermissionMiddleware({ organization: ["update"] })` . The

business always comes from the session; every query filters on it first; an id of another business, an unknown one, or the wrong kind is one `404 not_found`.

GET /settings/services `200 { canEdit, services, people, timezone }` (`timezone` from 12d.5, null before the business's hours): `services` by name, each `{ id, name, slug, description, durationMinutes, bufferBeforeMinutes, bufferAfterMinutes, slotIntervalMinutes, personChoice, asksAddress, active, peopleIds, placeIds }`; `people` every person and place `{ id, name, kind, active }` for the ticks (12d.3).

POST /settings/services, body the service schema. `201 { service }`. **PUT /settings/services/:serviceId**, the same body. `200 { service, outsideHours }` (`outsideHours` empty until 12d.5, and when the length did not change). **PUT /settings/services/:serviceId/resources**, `{ peopleIds: string[], placeIds: string[] }`, no repeats. `200 { peopleIds, placeIds }`.

GET /settings/people `200 { canEdit, people, timezone, senderDomain }`: every person and place by name, `{ id, name, kind, active, workEmail }` (`workEmail` from 12d.4, null for a place); `senderDomain` the business's sending domain or null (12d.4). **POST /settings/people**, `{ name, kind: "person" | "place" }`. `201 { resource }`. **PUT /settings/people/:resourceId**, `{ name, active, workEmail? }`. `200 { resource, upcomingBookings }`: the resource's upcoming confirmed bookings when it was just turned off, else `[]`; rows `{ bookingId, leadId, customerName, serviceName, personName, startsAt, endsAt }`, ISO, soonest first (12a's `outsideHours` row).

Errors: 400 `{ error: { code: "bad_request", message }, field }` from the first Zod issue; 409 `last_person` "Someone has to stay on: turn another person on first."; 409 `name_taken` "Another person or place here already has that name."; 409 `no_sending_address` "Set the business's sending address first."; 401/403 as 12a.

Formats: minutes as whole numbers; names trimmed, compared ignoring case for `name_taken`; work email trimmed and lowercased, its domain compared lowercased.

User text (names, descriptions) is rendered by React as text. Emails render the person's name through the existing templates' escaping.

Testing

Commands: `npm run test --workspace=@scheduleads-app/shared`, `npm run test --workspace=backend` (local Postgres, migrated and seeded), `npm run build` and `npm run lint` for the frontend. No Verify command and no browser test harness; hand checks are in each Done when.

One test per rule that would hurt if broken:

- 12d.1: a member without the permission is refused a save; another business's service gets the 404; a new service gets a unique slug and `month`, and a rename keeps the slug; a hidden service is no longer offered and its bookings stay.

- 12d.2: the last active person cannot be turned off; a name already taken is refused; a turned-off person is not offered and their bookings stay, listed in the answer.
- 12d.3: a save replaces the ticks; a place in `peopleIds`, or another business's person, is refused and nothing changes.
- 12d.4: a work email at another domain is refused, and none can be set without a sending address; the recipients rule: with a work email the confirmation replies to the person and the person is notified, without one nothing changes.
- 12d.5, named as on the Simulate page: a longer service that runs a booking past the end of the day is listed; a longer service that still fits is not listed; a shorter service never lists a booking; a booking already outside before the change is not listed again; a longer booking that would run into a gap between windows is listed; another service's bookings are never listed; a cancelled or past booking is never listed.

Notes for the AI

- 12a's patterns: client components fetching through `dashboardApiClient` with local state; react-hook-form with shadcn Input, Label, Button; each error under its field with `aria-describedby`; a refused save focuses the first bad field; saved and failed messages announced and cleared on the next edit. No TanStack Query (feature 14).
- Nothing is pre-filled where the business must choose (decision 30): who picks the person and whether it asks the address start unpicked; buffers start at 0 (none), start times start at "every service length".
- The setup command keeps working for a new business; it adds only what is missing, by name.
- Comments beside the line they explain, one or two lines, no history.

Open questions

None blocking. Every choice is in the build log's Decisions drawer and can be overruled at its step's plan; the two worth a look before their steps are decision 1 (a page per section) and decision 9 (what "no longer fits at the new length" means).

Implementation walkthrough

Feature 12d was specced heavy with five steps, one per real risk, and kept them: writing services, a person's or place's lifecycle, who does what, a work email that changes who gets which email, and the time rule for a service's new length. Every step had the builder's audit, a fresh reviewer and checks of the fixes until nothing new was found; three final reviews of the whole feature closed it. The Simulate page's seven length cases became saved tests under the same names in 12d.5. Settings became one page per section (decision 1).

frontend: Settings, one page per section (12d.1)

`app/(dashboard)/settings/` holds a shared layout with the section list (Hours, Services, People, the current one marked) and one page each; `/settings` opens Hours. Reading a section is shared by `settings-section.tsx` and `lib/api-client/settings/read-settings-response.ts`; 12a's notice and save result became `SaveNotice` and `SaveResultType`, used by every section.

services (12d.1, 12d.3, 12d.5)

`serviceValidationSchema` (shared, strict) checks a service once, the setup file included, since its length and buffer parts are the same schemas. `GET /settings/services` answers every service with its ticks, every person and place to tick (no work emails), and the business's time zone. `add-service.ts` gives a new service its slug once from its name (`toSlug`, then `freeSlug` for `-2`, `-3`; the unique index settles two adds at once) and the `month` layout; a rename never changes the slug. Hidden is `active = false`: the booking window stops offering it and its bookings stay.

Who does what (12d.3) is `PUT /settings/services/:id/resources`: the ticks replace the ones before in one transaction that locks the service's row, and every id must be a person or a place of this business in the right list, or the save is refused on that list. The booking side already read the ticks through `findServiceResources`. On the form, `ticks-field.tsx` shows the two lists, turned-off ones marked off, the plan's two hints, and a red hint when everyone ticked is off, since the booking side never falls back to "anyone" then. The form keeps one Save: the service, then its ticks when they changed; if the ticks fail, the form says so, the next Save changes the service it already made, and Cancel still hands the list the service as saved.

A new length (12d.5) changes no booking. The service save is one transaction that locks the service row and the business's hours row (`no key update`, so an hours save running meanwhile finishes first and bookings are not held up), reads each person's own hours (`findPeopleHours`, now shared with the hours save) and this service's upcoming bookings, and answers `outsideHours: applyLengthChangeRules` lists the confirmed bookings still to come that fit as booked and would not fit from the same start at the new length, by 12a's exported `fitsHours`. The Services page shows the list under the saved message, each booking linking to its lead; a save again at the same length keeps the last list, a new length replaces it.

people and places (12d.2, 12d.4)

`GET`, `POST` and `PUT /settings/people` add, rename and turn people and places off and on; never deleted (decision 3). Each save locks the business's row so two saves cannot both find a name free (refused ignoring case and outer spaces, decision 6) or both find someone else still on (the last person on cannot go off, decision 10). Turning one off answers their upcoming bookings, unchanged, in 12a's row shape (`ListedBookingType`, which also replaced 12a's outside-hours type, F-337). A person turned off still gets worker texts about the bookings they hold (F-344, on Frank's yes, reversing 8c's decision 1).

A work email (12d.4) is `resource.workEmail` (migration 0027, a person's only), stored trimmed and lowercased. It must be at the domain of the business's sending address, checked only when the address itself changes, so a rename still saves after a move of the sending domain (F-356); without a sending address none can be set. `findBookingEmailRecipients` decides who hears: the confirmation replies to the booked person's work email, and a new email kind, `booking_person_notification`, goes to it, also for bookings the owner made, never twice to the business's own address. `book-time.ts` queues that job only when the person has a work email, and the job reads it again when it runs. Moves, cancellations and reminders are unchanged (decision 7).

the setup command and the seed

Both now find the first person by the owner's login, so a rename on Settings never makes them add a second one (F-345, F-350), and tick only a service they make; an existing service's ticks are the owner's, and the report names a file tick missing there as "in the file, not ticked here" (F-354), while a tick naming nobody still stops the run (F-355).

shared naming

The resource schemas are one form per file (`add-resource-validation-schema.ts` , `save-resource-validation-schema.ts` , their name in `resource-name-validation-schema.ts`), and `freeSlug` is `helpers/free-slug.ts` , exported as `@scheduleads-app/shared/free-slug` (F-362).

Carried

F-364 (P3): nine comments added here carry step tags such as "(12d.4)"; dropped in the next feature's first commit rather than re-reviewing the whole feature for comments. F-348 stays a lead, as F-331. F-358 accepted by Frank. Two "Hand check" bookings of Exterior estimate remain in Summit's dev data.

Findings

12d/F-337 [P3] closed - OutsideHoursBookingType still sits in a file of its own, though outsideHoursOf now produces it

File: backend/lib/settings/outside-hours-booking-type.ts:1-11; backend/lib/settings/outside-hours-of.ts:15-24 **Found:** 2026-10-10 by the final review of feature 12a (scope: main...76850fe; lenses: quality, security, performance, tests) **Why it matters:** The type-only file made sense in 12a.1, when `outsideHours` was always empty and nothing produced it. Since 12a.2 `outsideHoursOf` builds every row of it, and coding-standards.md says "A type sits in the file of the function that produces it". The file is used (outside-hours-of, both saves), so nothing breaks; it is the leftover the standard exists to stop, a file Frank opens to find only a shape whose maker lives elsewhere. **Suggested fix:** Move `OutsideHoursBookingType` into outside-hours-of.ts, point the two saves' imports there, and remove outside-hours-booking-type.ts. **Resolution:** Fixed 2026-10-10 in step 12d.2: the type became

ListedBookingType in backend/lib/settings/listed-booking.ts, beside listedBookingOf, the function that produces it, used by outsideHoursOf (12a) and the People save (12d.2); outside-hours-booking-type.ts is gone. Closed 2026-10-10 by independent review of step 12d.2 (2a517f5..f1f8427): the old file and every import of it are gone; ListedBookingType sits beside listedBookingOf, used by outsideHoursOf, both hours saves, the services route and saveResource; backend tests, frontend build and lint pass.

12d/F-340 [P3] closed - A service renamed on Settings no longer matches its setup-file entry, so a setup run under the new name adds a second one

File: packages/shared/client-setup/apply-business-shape.ts:145-150; packages/shared/client-setup/run-client-setup.ts:112-121; backend/lib/settings/save-service.ts:20 **Found:** 2026-10-10 by independent review of step 12d.1 (scope: main...e18c11a; lenses: quality, security, performance, tests) **Why it matters:** The setup command finds a business's service by `toSlug(service.name)`, which held only while every slug came from the current name. 12d.1 keeps the slug fixed and lets the name change (decision 5), and gives a second add of a name `-2`. So after "Estimate" is renamed "Free estimate" on Settings, a setup file listing "Free estimate" inserts a second live "Free estimate", while one still listing "Estimate" keeps matching the renamed row; and a Settings-made `estimate-2` is taken for a setup service named "Estimate 2". The spec's note says the command "adds only what is missing, by name", which is not what it does once names change. Only re-runs on an existing business are affected, and those are local `*_dev` only until item 10b. **Suggested fix:** Match setup services by name, trimmed and ignoring case (as the spec says), in both apply-business-shape.ts and run-client-setup.ts, keeping `toSlug` only for the slug of a new row (with the same `-2` rule as `addService`); or state in the command's header that a re-run after Settings changes is unsupported. **Resolution:** Fixed 2026-10-10: the setup command finds a service by its name, ignoring case and spaces at the ends (client-setup/service-named.ts, used by the apply and by the differences report), never by slug; a service it adds gets a free slug through the same `freeSlug` rule as Settings (helpers/to-slug.ts), so a slug a renamed service still holds gets `-2`. New test in run-client-setup.test.ts, "finds a service renamed on Settings by its new name, never adding it twice", fails on the old matching. Closed 2026-10-10 by the check of 12d.1's fixes (6416063..0909c91): `serviceName` scopes to the business and compares `lower(btrim(name))` with the trimmed, lowercased file name (the local database lowers accented letters the same way as JS); the apply and both places in the differences report use it, and a new service's slug goes through the same `freeSlug` as `addService`. The new test fails on 6416063's apply-business-shape.ts and run-client-setup.ts ("expected ['1 services', ...] to deeply equal []"), passes on 0909c91, uses a random throwaway business and is removed with it. The one new gap, two services of the same name, is F-343.

12d/F-341 [P3] closed - Closing the service form drops keyboard focus to the top of the page

File: frontend/components/settings/services-screen.tsx:34-38, 58, 75 **Found:** 2026-10-10 by independent review of step 12d.1 (scope: main...e18c11a; lenses: quality, security, performance, tests) **Why it matters:** Save, Add and Cancel all `setOpen(null)`, which unmounts the form that holds the focus, and nothing moves it anywhere. A keyboard or screen reader user hears "Saved" from the status

region but is left at the document's start and has to tab back through the dashboard to reach the list. 12a's cards never unmounted, so this is new with the open-one-form pattern. **Suggested fix:** After closing, focus the row's Change button (by a ref map keyed by service id) for an edit, and "+ Add a service" for a new one or a Cancel. **Resolution:** Fixed 2026-10-10: closing the service form (save or cancel) returns the keyboard to the button that opened it: the row's Change button, or + Add a service after cancelling a new one. Checked in the browser for all three. Closed 2026-10-10 by the check of 12d.1's fixes (6416063..0909c91): close() stores the button id in a ref and the effect on `open` focuses it after the commit that removes the form; on a new service the same commit adds its row, so its Change button exists when the effect runs. Switching straight to another form never sets the ref, and that form focuses its own name field. Build and lint clean.

12d/F-342 [P3] closed - The start-times minutes box sits inside the "Every" radio's label

File: frontend/components/settings/service-form.tsx:304-330 **Found:** 2026-10-10 by independent review of step 12d.1 (scope: main...e18c11a; lenses: quality, security, performance, tests) **Why it matters:** HTML allows a label no labelable descendant other than its control; this label holds the radio and the number input, so the radio's name is computed with an embedded spinbutton, and the disabled box (shown while "every service length" is picked) is the natural thing to click but is disabled, so it may not pick the radio. Unverified: not tried in a browser or screen reader. **Suggested fix:** Close the label after "Every"; put the input beside it with its aria-label, and either leave it enabled and pick the radio on focus, or keep it disabled and say so in a hint. **Resolution:** Fixed 2026-10-10: the minutes box sits beside the Every choice, outside its label, and is always enabled; typing a number picks Every. Picking Every no longer fills in 30: the owner types the number. Checked in the browser. Closed 2026-10-10 by the check of 12d.1's fixes (6416063..0909c91): the label now holds only the Every radio; the box beside it keeps its aria-label and is never disabled; any typed value makes the field non-null, so Every is checked; picking Every sets NaN, which the schema rejects ("Use whole minutes.") until a number is typed, as an emptied box already did.

12d/F-343 [P3] closed - With two services of the same name, the setup report lists the other one's ticks as "ticked by hand"

File: packages/shared/client-setup/run-client-setup.ts:186-201; packages/shared/client-setup/service-named.ts:8-13 **Found:** 2026-10-10 by the check of 12d.1's fixes (scope: 6416063..0909c91) **Why it matters:** Settings allows a second service with an existing name (addService gives it `-2`), and a rename can make two names equal. The apply and the field comparison take the oldest such service (`orderBy(createdAt).limit(1)`), but the "ticked by hand" query has no limit, so it gathers the ticks of every same-named service. Run on a throwaway business: setup made "Estimate" ticked Ana, a second "Estimate" (estimate-2) was ticked Bob, and a dry run reported "Estimate", `ticked by hand: Bob`, though Bob is not on the service the setup manages. Under the old slug matching estimate-2 was never matched. It only misleads the report (nothing is written), local `*_dev` only until item 10b. Also, `createdAt` alone has no tiebreak, so two same-named rows made in one transaction (the seed) have no fixed order. **Suggested fix:** Find the service once per file entry (order by createdAt, then id) and use that

row's id in the ticks query, instead of matching by name again. **Resolution:** Fixed 2026-10-10: the ticks report first resolves the one service the file means (by name, oldest first, then by id), then reads that service's ticks by its id, so another of the same name never lends it its ticks; every by-name lookup orders by createdAt then id (oldestServiceFirst in service-named.ts). No new test: only a report line is at stake; the setup tests still pass. Closed 2026-10-10 by the independent review of feature 12d (e1a5d8f..7d2dfe2): run-client-setup.ts:206-222 resolves the file's service once (serviceNamed, oldestServiceFirst: createdAt then id) and reads only that id's ticks; the field differences (126-131) and the apply (apply-business-shape.ts:153-158) use the same order, so all three mean the same row; shared 174 of 174.

12d/F-344 [P2] closed - A person turned off is no longer texted about the bookings they still hold, so a move or cancellation of one never reaches them

File: backend/lib/text/send-worker-text.ts:148; backend/lib/settings/save-resource.ts:44-63 **Found:** 2026-10-10 by independent review of step 12d.2 (scope: 2a517f5..f1f8427; lenses: quality, security, performance, tests) **Why it matters:** sendWorkerText returns "the person is inactive" before every kind, the "off your day" (**removed**) text included. Before 12d.2 nobody could turn a person off from the app; now the step's own story turns Tomas off and keeps his Friday booking. If that customer then cancels, or moves (the move goes to someone still on, since Tomas is no longer offered), Tomas's "off your day" text is skipped and he still believes he has Friday. The spec and the form's copy say off never touches a booking and the ones already made stay; the worker's loop on those bookings silently stops. worker-text-job.test.ts:367 tests the old rule ("an inactive person gets nothing"), written when inactive could only be set by hand. **Suggested fix:** Let **removed** (and arguably **moved**) texts through for an inactive person who still holds or held the booking, keeping **added** off; or tell the owner on the turned-off answer that the person will no longer get texts about the listed bookings. Frank's call which. **Resolution:** Fixed 2026-10-10 on Frank's yes ("yeah"): a person turned off still gets worker texts about the bookings they hold, by their own text settings; off only stops new bookings. This reverses feature 8c's decision 1 (an inactive person gets no texts), made when only a hand edit could turn someone off. send-worker-text.ts no longer skips an inactive person; find-worker-text-settings.ts no longer reads active. The 8c test became "a person turned off still hears about a booking they hold". Closed 2026-10-10 by the check of 12d.2's fixes (db01a23..edf075d): sendWorkerText was the only reader of the dropped **active** field (findWorkerTextSettings has no other caller; its join to resource still keeps the read inside the business). Every kind stays sensible for someone off: whyNotDue still sends **added** and **moved** only while the booking is theirs, and a move off them cannot pick an off person (move-booking.ts:106 checks the offered list), so they get the **removed** text; their own three switches still apply. The changed job test expects one text and fails on the old line, which sent none.

12d/F-345 [P2] closed - The setup command finds people by exact name, so a rename on Settings breaks it: the first person renamed stops the command, others are added twice

File: packages/shared/client-setup/run-client-setup.ts:47-55, 143-147; packages/shared/client-setup/apply-business-shape.ts:29-44 **Found:** 2026-10-10 by independent review of step 12d.2 (scope:

2a517f5..f1f8427; lenses: quality, security, performance, tests) **Why it matters:** runClientSetup finds "the first person" by `resource.name = organization.name`; resource-table.ts:18 says the owner renames that person in feature 12, and 12d.2 is that rename. After it, every run of the command for that business answers "`<business>`" has no first person named after it. and stops. ensureResource matches the file's people with `eq(resource.name, name)`, case and spaces exact, while Settings now enforces decision 6 (case and outer spaces ignored): a person renamed "Diego (painter)" to "Diego" is inserted again under the old name by a file that still lists it, and a file listing "diego" next to a Settings "Diego" adds a second one the People page would have refused. Same class as F-340 for services. Local `_dev` only until item 10b. **Suggested fix:** Find the first person by the business's owner login (`resource.userId` of the owner member) rather than by name, and match the file's people with the same `lower(btrim(name))` rule as `nameTaken` (one shared helper, as `service-named.ts` does for services), in `ensureResource` and the differences report. **Resolution:** Fixed 2026-10-10: the setup command finds the first person by their tie to a login (the person made with the business), not by the business's name, and ticks may name them by either; every person and place is matched by name ignoring case and spaces at the ends (`client-setup/resource-named.ts`, `name-key-of.ts`, shared with `service-named.ts`), in the apply, the differences report and the ticks report. The test fixture now gives the first person a login, as every real business has. New test "finds people renamed on Settings, the first one by its login, never adding them twice" fails on the old code. A dry run of `client-setups/agentsweb.ts`: nothing to add. Closed 2026-10-10 by the check of 12d.2's fixes (`db01a23..edf075d`): only the first person ever gets a `userId` (the `afterCreateOrganization` hook in `auth-server.ts`, migration 0004's backfill, the seed; `add-resource.ts` and `ensureResource` never set one), and it is the business's oldest resource, so `isNotNull(userId)` oldest first finds it. A business left unlinked by 0004 (no owner or two, or first person already renamed) or whose owner login was deleted (`on delete set null`) now gets "no first person tied to a login" where the name lookup used to work; none is known, and the command is local `_dev` only until item 10b, so left as a note. People are matched with `lower(btrim())` in `ensureResource` and the differences report, ticks by `nameKeyOf`; the new test passes, and the fixture's users are deleted in `afterAll`. One gap left in the ticks report after the first person is renamed: F-349. Dry run of `agentsweb.ts`: nothing to add.

12d/F-346 [P3] closed - "That name is taken" shows as a form alert, not under the Name field

File: frontend/lib/api-client/settings/read-save-refusal.ts:15; backend/routes/settings-routes.ts:200, 226 **Found:** 2026-10-10 by independent review of step 12d.2 (scope: 2a517f5..f1f8427; lenses: quality, security, performance, tests) **Why it matters:** `name_taken` is a 409 with no `field`, and `readSaveRefusal` makes a field error only from a 400, so both People forms show it in the `SaveNotice`: the Name input is not marked `aria-invalid`, has no error under it, and does not take the focus, unlike every other field error in the 12a pattern the spec names ("each error under its field ... a refused save focuses the first bad field"). **Suggested fix:** Return `field: "name"` with the 409 and let `readSaveRefusal` read a field from any refusal that carries one, or map `name_taken` to the name field in `save-resource.ts` on the frontend. **Resolution:** Fixed 2026-10-10: the 409 `name_taken` answer carries field "name", and the frontend reads a field on any refusal, so the message shows under Name with focus there. Checked in the browser: adding "room

2" when Room 2 exists. Closed 2026-10-10 by the check of 12d.2's fixes (db01a23..edf075d): readSaveRefusal's callers are only the settings saves (business hours, person hours, people, services), and in settings-routes.ts only refuseFirstIssue (400) and NAME_TAKEN (409, People add and change only) carry a `field`; no_business_hours, not_found and last_person carry none, nor does any middleware. leads-routes.ts's `field` answers are 400s and do not go through readSaveRefusal. Both People forms register "name", so the field always exists. The route test now expects `field: "name"`.

12d/F-347 [P3] closed - A place turned off listing the bookings that use it has no test

File: backend/lib/settings/find-upcoming-bookings.ts:67; backend/lib/settings/save-resource.ts:70; backend/routes/settings-routes.test.ts:558-562 **Found:** 2026-10-10 by independent review of step 12d.2 (scope: 2a517f5..f1f8427; lenses: quality, security, performance, tests) **Why it matters:** The Built note adds this rule ("a place turned off lists the bookings that use it") with its own query branch on booking.placeId; the only place test turns off a place with no bookings and checks the 200. A slip (person id passed, wrong column) would list nothing, and the owner would believe nobody needs a call. **Suggested fix:** In the turned-off test, give one booking a placeId, turn that place off, and expect that booking (and not a booking without the place) in `upcomingBookings`. **Resolution:** Fixed 2026-10-10: new route test "a place turned off lists the bookings that use it": a booking in Bay 1 is listed, one without a place is not; it fails without the place branch in find-upcoming-bookings.ts. Closed 2026-10-10 by the check of 12d.2's fixes (db01a23..edf075d): with the placeId condition replaced by `undefined`, the test failed (two bookings listed, one expected) and passed again once restored; makeBooking's placeId defaults to null, so its other callers are unchanged.

12d/F-349 [P3] closed - After the first person is renamed, the setup report lists them as "ticked by hand" on every service the file ticks them on by the business's name

File: packages/shared/client-setup/run-client-setup.ts:64, 68, 218-221 **Found:** 2026-10-10 by the check of 12d.2's fixes (scope: db01a23..edf075d) **Why it matters:** The F-345 repair lets a tick name the first person by the business's name or their own (`names: [business.name, firstPerson.name]`), but only in the apply; findDifferences gets no first person, so its ticks report compares the saved person's name with the file's tick names alone. Reproduced with a throwaway test (removed after): a file ticking "Phone call" with the business's name, applied, the first person renamed "Owner", then a dry run answered `differences: [ "\"Phone call\"", ticked by hand: Owner ]`, though the file put that tick there and the apply made nothing. Before the repair the command stopped on the rename, so this report never ran. Only a report line misleads; nothing is written; local `*_dev` only until item 10b. **Suggested fix:** Pass the first person's id (or the same names list) to findDifferences and treat a tick on that id as in the file when any of the file's ticks matches one of its names; or compare ticks by resource id, mapping the file's names to ids as the apply does. **Resolution:** Fixed 2026-10-10: the differences report knows the first person and the business's name; a tick of the first person counts as in the file when the file ticks them by their own name or by the business's. The setup tests pass; no new test, only a report line is at stake. Closed 2026-10-10 by the independent review of feature 12d (e1a5d8f..7d2dfe2): findDifferences gets the first person's id and the business's name (run-client-setup.ts:64-67); a tick on that id counts as in the file

when the file names them by either name (224-226), and the "in the file, not ticked here" side adds the business's name when they are ticked (230-233), matching the apply's name map (apply-business-shape.ts:93-95).

12d/F-350 [P3] closed - The seed, run again after the first person is renamed on Settings, adds a second one and stops on the login index

File: packages/shared/scripts/seed-dev.ts:440-447; packages/shared/db/booking-tables/resource-table.ts:35 **Found:** 2026-10-10 by the check of 12d.2's fixes (scope: db01a23..edf075d) **Why it matters:** The seed still finds its first person by the business's name (ensureResource), the lookup F-345 replaced in the setup command. Rename "Summit Painting (dev)" on People (12d.2) and reseed: ensureResource finds nobody of that name and inserts a new person, then the link update sets the owner's userId on it (`isNull(userId)` holds for the new row), which the unique index on (organizationId, userId) refuses, since the renamed person already holds that login; the whole seed transaction rolls back. Read from the code, not run (the seed is outside this check's commands). The seed is re-runnable by design ("no machine needs a rebuild"); a dropped database is unaffected. **Suggested fix:** Find the seed's first person by the owner's login first (`resource.userId = userId` in that business), falling back to ensureResource by name only when none is linked yet; pass that person's current name with the business's name to applyBusinessShape, as run-client-setup.ts now does. **Resolution:** Fixed 2026-10-10: the seed finds a business's first person by their owner's login once linked, and only falls back to the business's name to make one; a rename on Settings no longer makes it add a second. Ran npm run db:seed on scheduleads_dev: both businesses "already there". Closed 2026-10-10 by the independent review of feature 12d (e1a5d8f..7d2dfe2): seed-dev.ts:441-448 looks the person up by (business, owner userId), unique by resource_organization_user_unique, and only falls back to ensureResource by name when none is linked, so a renamed first person is found and the link update (452-455) is a no-op; ticks by the business's name still map to that id (names: [business.name]). Read, not run: the seed is outside this review's commands.

12d/F-351 [P3] closed - Cancel after a service saved but its ticks failed leaves the list showing the service as it was

File: frontend/components/settings/service-form.tsx:184-186 **Found:** 2026-10-10 by /audit (scope: 12d.3, 7eb80af..7f38977; lenses: all) **Why it matters:** The form saves the service, then its ticks by a second call. When the second call fails, the form stays open with the service already changed on the server; Cancel on an existing service called onCancel, so the list kept the old name, length or Live state until a reload. (A new service was already handled.) **Suggested fix:** On Cancel, hand the list the service as last saved whenever a save landed in this form. **Resolution:** Fixed 2026-10-10: Cancel passes the last saved service to the list whenever one landed, new or existing. Closed 2026-10-10 by the check of 12d.3's fixes (f38b317..397e536): savedService only moves off the opened service when the service saved and its ticks then failed (every other save closes the form), so Cancel hands the list exactly that case; a failed first save still cancels plainly.

12d/F-352 [P2] closed - The step's hand check left Summit's Colour consultation on "the customer picks", and three public route tests now fail

File: backend/routes/public-booking-links-routes.test.ts:226, 605, 619 **Found:** 2026-10-10 by the independent review of 12d.3 (scope: 7eb80af..7c8e9e3) **Why it matters:** These tests read painting-dev's seeded Colour consultation and expect "the business sends whoever is free". The step's hand check switched it to "the customer picks" in scheduleads_dev and left it there, so `npm run test --workspace=backend` now ends 3 failed, 924 passed ("expected 'customer_picks' to be 'business_assigns'", people listed where none are expected, 200 where 400 is expected). Every later step reruns this suite, so the gate is red for reasons that have nothing to do with the code. `npm run db:seed` does not put it back: applyBusinessShape only adds a missing service and never changes an existing one. **Suggested fix:** Switch Colour consultation back on Settings (or drop, migrate and seed scheduleads_dev), rerun the backend suite, and do future hand checks on a service the suite does not read, or put the change back as part of the check. **Resolution:** Fixed 2026-10-10: Colour consultation switched back to "we send whoever is free" on Settings; backend 927 of 927 again. Dev data only, no code. Closed 2026-10-10 by the check of 12d.3's fixes (f38b317..397e536): `npm run test --workspace=backend` 927 passed, 0 failed.

12d/F-353 [P3] closed - Cancel after the ticks failed to save announces "Saved. Customers can book it now."

File: frontend/components/settings/service-form.tsx:185-188, frontend/components/settings/services-screen.tsx:47-58 **Found:** 2026-10-10 by the independent review of 12d.3 (scope: 7eb80af..7c8e9e3) **Why it matters:** When the service saves and its ticks then fail, the form says "X is saved, but who does it is not". If the owner presses Cancel, the form (since the F-351 repair, and for a new service since 7f38977) calls onSaved, and the list's saved() shows the same notice as a full save: "Saved. Customers can book X now." The owner who wanted only Marco reads that everything went through, while the service is live with the ticks as they were (for a new one, nobody ticked, so anyone is offered). Read from the code, not clicked. **Suggested fix:** Let Cancel hand the list the saved service without the success notice (a second callback, or a flag on onSaved), or have the list say that who does it was not changed. **Resolution:** Fixed 2026-10-10 on Frank's yes: Cancel after a failed tick save tells the list so, and the list says "Saved, but who does <service> was not." instead of the full-save notice. Checked in the browser with the ticks call blocked on Cabinet consultation. Closed 2026-10-10 by the check of 12d.3's fixes (f38b317..397e536): Cancel is the only caller passing `false`, every full save still passes nothing (true), and the list returns before the success notice; frontend build and lint clean.

12d/F-354 [P3] closed - The setup command and the seed put back ticks the owner removed on Settings

File: packages/shared/client-setup/apply-business-shape.ts:194-199 **Found:** 2026-10-10 by the independent review of 12d.3 (scope: 7eb80af..7c8e9e3) **Why it matters:** For a service that already exists, the setup still inserts every tick the file names (`onConflictDoNothing`, "one removed by hand comes

back"). That rule dates from when "by hand" meant editing the table; since 12d.3 it is the owner's own choice on Settings. On Riverbend Clinic (dev), untick Mei from the laser service, then run `npm run db:seed` (or the setup with `--apply`): Mei is ticked and offered again; a service whose ticks were cleared to "anyone" goes back to only the file's people. The setup already leaves an existing service's other fields alone, so ticks are the odd one out. Read from the code, not run; the setup is local `*_dev` only until item 10b. **Suggested fix:** Add a service's ticks only when this run makes the service, and list the file's missing ticks on an existing service as a difference instead; Frank's call, since the current rule was written on purpose. **Resolution:** Fixed 2026-10-10 on Frank's yes: the setup and the seed tick only a service they make now; an existing service's ticks are left as the owner set them, and the report names a file tick missing there as "unticked by hand". The setup test unticks Room A and checks it stays off and is reported; it fails on the old code. Closed 2026-10-10 by the check of 12d.3's fixes (f38b317..397e536): a service made now still gets the file's ticks through the same name map (the first person by the business's name in both callers, and by their own name in the setup), an existing one is skipped before any insert, and the report names added and removed ticks, the first person included; shared 174 of 174. The skip also took the "who is not listed" guard with it for existing services, see F-355.

12d/F-355 [P3] closed - On a service that already exists, a tick naming nobody is no longer stopped, and is reported as "unticked by hand"

File: packages/shared/client-setup/apply-business-shape.ts:189-192; packages/shared/client-setup/run-client-setup.ts:229-234 **Found:** 2026-10-10 by the check of 12d.3's fixes (scope: f38b317..397e536) **Why it matters:** The F-354 skip (`if (existingLink || !ticked.length) continue`) sits before the map that throws `"X" ticks "Y", who is not listed.`, so that guard now runs only for a service made in this run. A setup file whose existing "Laser" ticks a misspelled "Mai" used to stop the run with that message; now the dry run and the apply both pass, and the report says `"Laser", unticked by hand: Mai`, which reads as the owner's own choice on Settings. The same line appears for a person the file adds in this very run and ticks on an existing service (made, never ticked, reported as removed by hand). Nothing wrong is written; only the check and the wording are lost. Read from the code, not run; local `*_dev` only until item 10b. **Suggested fix:** Resolve every service's tick names (the throw) before the `existingLink` skip, so a name nobody holds stops the run as before; and word the report line for a file tick missing on an existing service without claiming "by hand" (for example "in the file, not ticked"), or name a person made in this run apart. **Resolution:** Fixed 2026-10-10 (it only made F-354 hold): every tick name is checked before an existing service is skipped, and the report line reads "in the file, not ticked here". A test runs a misspelt tick on an existing service and expects the stop; it fails on 397e536. Closed 2026-10-10 by the check of 12d.3's fixes (397e536..aeeb451): the name map and its throw now run for every service before the `existingLink` skip, inside the transaction, so a dry run and an apply both stop and roll back; a service made now still gets the file's ticks (the first person by the business's name in both callers); the report line no longer says "by hand"; shared 174 of 174, backend 927 of 927.

12d/F-356 [P3] closed - Changing the business's sending address to another domain leaves work emails at the old one

File: backend/lib/email/save-email-sending.ts; backend/lib/settings/save-resource.ts **Found:** 2026-10-10 by /audit (scope: 12d.4, 0701f77..a8f83a6; lenses: all) **Why it matters:** A work email is checked

against the sending domain only when it is saved (decision 8). If the business later moves its sending address to another domain (feature 6's save), the people's work emails stay at the old one and keep getting the person's notification and the customer's replies. Nothing breaks and nobody outside the business is reached, but the rule "at the business's own domain" stops holding until each is re-saved.

Suggested fix: When the sending domain changes, list the people whose work email is at another domain (or clear them), on the email settings save; or accept it, since a domain move is rare and done with the agency.

Resolution: Confirmed at P3 by the independent review of 12d.4 (0701f77..b170bbf). `save-email-sending.ts:78` writes `senderEmail` and never reads `resource.workEmail` (it is read nowhere but the settings and email code), and `send-booking-emails.ts` sends to and replies to the stored address without re-checking its domain. One more effect, found in the form: `change-resource-form.tsx:43-46` always sends a person's stored work email, so after a domain move any save of that person (a rename, or turning them off) re-runs the check at `save-resource.ts:53-68` and is refused 400 "Use an address at <new domain>" until the owner changes or clears the field. The message sits on the field and is focused, so the owner is not stuck, and that refusal is also where the stale address first shows. Still open. Fixed 2026-10-10 on Frank's yes: the domain is checked only when the work email itself changes, so after a move of the sending domain a rename or turning off still saves and the old address keeps working until replaced; a new one must be at the new domain. No work email is cleared automatically. A route test covers it and fails on 0a78fea. Closed 2026-10-10 by the check of 12d.4's fixes (b170bbf..05f2dc0): save-resource.ts is the only code that writes a work email, always through the trimmed, lowercased schema and the domain check, so an unchanged address can only be one that passed the check before a domain move (the agreed case); case or spaces in a resend compare equal and keep it, a person with none and any new address are still checked, a place is refused before the comparison, and the sending address can never be cleared back to none; backend 934 of 934.

12d/F-357 [P3] closed - The Services page's people list now carries every person's work email

File: backend/lib/settings/find-services-settings.ts:45; backend/lib/settings/resource-settings-of.ts:19

Found: 2026-10-10 by the independent review of 12d.4 (scope: 0701f77..b170bbf) **Why it matters:**

`findServicesSettings` reads the people for the ticks with `resourceSettingsColumns`, which 12d.4 widened with `workEmail`, so `GET /settings/services` now answers `people` as `{ id, name, kind, active, workEmail }`. The contract in current-feature.md (Data / contracts) says `{ id, name, kind, active }` for the ticks. Only members of the business can read it, and they already see the same addresses on `GET /settings/people`, so nothing leaves the business; it is a contract drift and an address sent where nothing uses it. **Suggested fix:** Give the ticks their own columns (id, name, kind, active), or amend the contract to say the Services answer carries `workEmail` too. **Resolution:** Fixed

2026-10-10 without asking (it only makes the contract hold): the Services read selects its own four columns, so the ticks carry no work email; a route test checks it and fails on b170bbf. Closed 2026-10-10 by the check of 12d.4's fixes (b170bbf..05f2dc0): the generic resourceSettingsOf returns only the columns it is given; People's read, add and save still pass resourceSettingsColumns and are typed ResourceSettingsType with workEmail, Services passes its own four, and no other backend read returns a work email; frontend build and lint pass.

12d/F-358 [P3] accepted - A confirmation or person's email retried after the work email changed reuses its Resend key with a different email

File: backend/lib/email/send-booking-emails.ts:54; backend/lib/email/send-booking-emails.ts:104; backend/lib/email/find-booking-email-context.ts:54 **Found:** 2026-10-10 by the independent review of 12d.4 (scope: 0701f77..b170bbf) **Why it matters:** The confirmation's `reply_to` and the person's notification's `to` come from `resource.workEmail` read afresh on every attempt, while each keeps one idempotency key (`booking-confirmation/<id>` , `booking-person-notification/<id>`). The code already guards this invariant elsewhere ("Resend refuses a key it already used with a different email", the confirmation's `stampedAt`). If an attempt reaches Resend but the app sees a failure (the 10 second timeout in `send-email.ts`), and the owner changes or sets that person's work email before the retry, the retry is refused by Resend, so the job fails on every retry and no timeline entry is written, although the email went. Narrow: it needs a timed out send and an edit inside the retry window; no customer is harmed. Read from the code, not run. **Suggested fix:** Accept it as rare, or freeze the address for the booking: keep the address a booking's emails were first built with (for example on the booking at booking time) so every retry sends the very same email. **Resolution:** Accepted 2026-10-10 by Frank: it needs Resend to send yet not answer within 10 seconds (about 1 send in 10,000) and the owner to change that person's work email in the ~3 seconds before the first retry (about 1 in 3 million), so about 1 booking in 30 billion; the email still reaches the person, only the log line and the timeline note are lost.

12d/F-359 [P2] closed - Saving again after the ticks failed loses the list of bookings the new length no longer fits

File: frontend/components/settings/service-form.tsx:150, 162; backend/lib/settings/save-service.ts:42-44 **Found:** 2026-10-10 by the independent review of 12d.5 (scope: 4fe1fe5..f7dc97b) **Why it matters:** The form saves the service first and its ticks second. When the length changes and the ticks then fail (the API not answering, a 5xx, a lapsed session: the path F-351 and F-353 already handle), the form stays open and the first answer's list sits in `outsideHours.current` . Pressing Save again calls `saveService` again with the same values; the backend now finds the length unchanged (60 to 90 was already saved) and answers `outsideHours: []` , and line 162 overwrites the held list with it. The page then says "Saved." with no list, so the owner is never told Maria's Tuesday 4:00 now runs past 5:00, which is the whole point of the step. Read from the code, not run. **Suggested fix:** Keep the list across saves of one open form: on a later save, merge the new answer's `outsideHours` into the held one by `bookingId` instead of replacing it (or, when the service values are unchanged since the last successful save, retry only the ticks). **Resolution:** Fixed 2026-10-10 without asking (it only makes the plan hold): the form keeps

every save's list, merged by booking, while it is open; checked in the browser with the ticks call blocked on the first save, Marta still listed after the second. Left fixed by the independent review of feature 12d (e1a5d8f..7d2dfe2): the original case now holds (a second save with the same length keeps the first list), but the merge never drops an entry, so a second save that puts the length back lists bookings that fit again: F-361. Closes once the merge applies only when the length is unchanged. Closed 2026-10-10 by the independent review of feature 12d (e1a5d8f..54dd2dd): service-form.tsx:165-168 keeps the held list when a save comes back at the length last saved by this form, which is the case here (the backend answers `[]` for an unchanged length, save-service.ts:42-44), so the first save's list survives a second save after the ticks failed.

12d/F-360 [P3] closed - A person's hours saved at the same moment as a new service length are not waited for, though the comment says hours saves are

File: backend/lib/settings/save-service.ts:46, 56, 60; backend/lib/settings/save-person-hours.ts:62, 74
Found: 2026-10-10 by the independent review of 12d.5 (scope: 4fe1fe5..f7dc97b) **Why it matters:** The service save reads the business's hours row `for share` so that "a save of hours running meanwhile finishes first". That holds for the business's own save, which takes the row `for update`. A person's save also takes the business row only `for share`, and two share locks never wait for each other, so `findPeopleHours` (unlocked) reads that person's old row while their save is still open. Example: Ana's Tuesday goes from ending 6:00 to ending 5:15 in one tab while the estimate goes from 60 to 90 in another; Maria's 4:00 fits both of Ana's weeks as booked, so the person save lists nothing, and the service save checks 4:00 to 5:30 against the old 6:00 and lists nothing either. No deadlock: neither hours save touches `booking_link`. Narrow (two owner saves of one business at once, the F-331 family); the comment overstates what the lock gives. **Suggested fix:** Read the business row `for("no key update")` instead of `for("share")` in the service save: it waits for a person's save holding it shared, and nothing takes a key share on that row, so bookings are not held up. Or keep the lock, correct the comment to "a save of the business's hours", and accept the gap with F-331. **Resolution:** Fixed 2026-10-10 without asking (it only makes the comment and the plan hold): the service save locks the business's hours row for no key update, so it waits for a person's save too, and bookings are not held up. Closed 2026-10-10 by the independent review of feature 12d (e1a5d8f..7d2dfe2): save-service.ts:57 takes the business row `for no key update`, which conflicts with the person save's `for share` (save-person-hours.ts:62) and the business save's `for update`, so either hours save finishes first; no cycle (the service save holds only its `booking_link` row, which neither hours save touches), and no booking path locks that row; backend 942 of 942.

12d/F-361 [P3] closed - Saving again after the length went back keeps listing bookings that fit again

File: frontend/components/settings/service-form.tsx:164-168 **Found:** 2026-10-10 by the independent review of feature 12d (scope: current, e1a5d8f..7d2dfe2; lenses: quality, security, performance, tests) **Why it matters:** The F-359 repair merges every save's `outsideHours` into the held list while the form is open and never drops an entry. The backend recomputes the list against the length just saved whenever

the length changes (save-service.ts:42-71), so a later answer is the whole truth for that length. Example: the estimate goes from 60 to 90, Maria's Tuesday 4:00 is listed, the ticks call fails and the form stays open; the owner puts the length back to 60 and saves; the backend answers `[]` (4:00 to 5:00 fits), the merge keeps Maria, and the page says "One booking no longer fits at the new length" and lists her under "bookings that no longer fit", though at 60 she fits. Read from the code, not run. Narrow: needs a failed ticks save first; nothing is written, the owner may only call a customer for nothing. **Suggested fix:** Merge only when the length is the one last saved: if `values.durationMinutes` differs from `savedService.current.durationMinutes`, replace the held list with the new answer (the backend's list is complete for the new length); keep it only when the length did not change, which is the F-359 case. **Resolution:** Fixed 2026-10-10 without asking (it only makes F-359's repair hold): a save at a new length replaces the held list, and only a save again at the same length keeps it; checked in the browser, 60 to 90 with the ticks blocked then back to 60 lists nothing. Closed 2026-10-10 by the independent review of feature 12d (e1a5d8f..54dd2dd): service-form.tsx:165-168 replaces the held list whenever the saved length differs from the last one this form saved (the first save always replaces, `lastSavedLength` starts null), and the backend's answer for a changed length is complete because it checks each booking as booked against the new length (apply-length-change-rules.ts:26-38); 60 to 90 to 60 and 60 to 90 to 120 both end with the right list. No new defect; a failed service save leaves the held list and `lastSavedLength` untouched.

12d/F-362 [P3] closed - Two new exports sit in files named for something else

File: packages/shared/zod-validation/resource-validation-schemas/resource-validation-schema.ts:12, 29; packages/shared/helpers/to-slug.ts:17 **Found:** 2026-10-10 by the independent review of feature 12d (scope: current, e1a5d8f..7d2dfe2; lenses: quality, security, performance, tests) **Why it matters:** coding-standards.md says Zod schemas are one object per form with the file named after it (`signInEmailValidationSchema` in `sign-in-email-validation-schema.ts`), and shared code is one file per export. `addResourceValidationSchema` and `saveResourceValidationSchema` are two forms in `resource-validation-schema.ts`, a name that matches neither, so the import does not lead to the file. `freeSlug`, a separate helper the backend and the setup command both use, was added inside `to-slug.ts`. Older files do group a form's field parts (weekly-hours, availability-rule), which `service-validation-schema.ts` follows; these two are different forms and a different helper. Naming only, no behaviour. **Suggested fix:** Split into `add-resource-validation-schema.ts` and `save-resource-validation-schema.ts` (the shared name schema in one, imported by the other, or its own file), and move `freeSlug` to `helpers/free-slug.ts`, exported through the helpers index as now. **Resolution:** Fixed 2026-10-10 without asking (naming the standards already set): the two forms are `add-resource-validation-schema.ts` and `save-resource-validation-schema.ts`, their shared name schema `resource-name-validation-schema.ts`; `freeSlug` is `helpers/free-slug.ts`, exported as `@scheduleads-app/shared/free-slug`. Closed 2026-10-10 by the independent review of feature 12d (e1a5d8f..54dd2dd): `add-resource-validation-schema.ts`, `save-resource-validation-schema.ts` and `resource-name-validation-schema.ts` each hold the one export they are named for, all three re-exported from `zod-validation/index.ts`; `freeSlug` alone in `helpers/free-slug.ts` with its own subpath export (package.json:23-26), the same one-helper-one-subpath shape as every other helper there (there is no helpers index; `./helpers` points at `to-slug`); `to-`

slug.ts holds only toSlug; both callers (add-service.ts:9, apply-business-shape.ts:20) import the new paths; the backend and frontend builds and all three test suites pass. No behaviour changed.

12d/F-363 [P3] closed - Code comments added by this feature cite finding numbers, which the ledger reuses after the merge

File: frontend/components/settings/service-form.tsx:164; backend/lib/settings/save-resource.ts:54; backend/lib/settings/save-service.ts:47; backend/lib/text/send-worker-text.ts:148; backend/routes/settings-routes.test.ts:766, 777; backend/lib/jobs/worker-text-job.test.ts:368; packages/shared/client-setup/run-client-setup.test.ts:223, 246 **Found:** 2026-10-10 by the independent review of feature 12d (scope: current, e1a5d8f..54dd2dd; lenses: quality, security, performance, tests) **Why it matters:** coding-standards.md (Comments) says "No history in code comments (step numbers, finding numbers ...): that lives in the build log." This feature adds nine comments ending in a bare (F-344), (F-356), (F-357), (F-354), (F-355), (F-359), (F-360), (F-361); the base had four (F-06, F-304, F-305). A bare ID is scoped to the live ledger: /complete archives these as 12d/F-359 and resets the ledger, which then starts at F-01 again, so after the merge (F-359) in service-form.tsx names nothing and later names an unrelated finding. The sentences themselves already carry the reason; only the tag is the problem. Naming only, no behaviour. **Suggested fix:** Drop the (F-NNN) tags (and the 12d, beside them in send-worker-text.ts:148 and worker-text-job.test.ts:368) from the nine comments, keeping each sentence; the link from a finding to its code stays in the ledger's File line and the build log. **Resolution:** Fixed 2026-10-10 without asking (the standards already forbid it): the nine tags are gone and the sentences kept; the older F-304 and F-305 tags in the setup files predate this feature and are left. Closed 2026-10-10 by the independent review of feature 12d (e1a5d8f..9417aa7): 54dd2dd..9417aa7 changes only those nine comment lines in seven code files, plus the ledger; each keeps its sentence and drops the tag, and no F-NNN remains on any code line this feature adds; all three test suites, both builds and the frontend lint pass. No new defect from the repair; the step-number tags in other comments of this feature are F-364.

Independent review

Status: passed **Target commit:** 9417aa7c1a0e42d9a31caac2eb72b007fec7c7e5 **Base commit:** e1a5d8f0885a91edd7134333a8d7cbab7cfc4001 **Base ref:** main **Spec hash:** 7b2749ae1a7dee4e9b436a666ec242151dac58c8885ce33f56fa3a224ee1bed1 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-11T00:03:35Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-11T00:07:31Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

`e1a5d8f0885a91edd7134333a8d7cbab7cfc4001..9417aa7c1a0e42d9a31caac2eb72b007fec7c7e5` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `git rev-parse HEAD` : pass, equals Target commit
- `git merge-base main 9417aa7` : pass, equals Base commit
- `sha256sum blueprint/context/current-feature.md` : pass, equals Spec hash
- `git status --short` : pass, only review.md and findings.md changed, plus the untracked blueprint/ai-voice-proposal.md (see Remaining risk)
- `npm run test --workspace=@scheduleads-app/shared` : pass, 174 of 174 (23 files)
- `npm run test --workspace=backend` : pass, 942 of 942 (83 files), against the local scheduleads_dev
- `npm run test --workspace=@frankdmosquera/booking-component` : pass, 79 of 79 (16 files)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass, /settings/hours, /settings/services and /settings/people built
- `npm run lint --workspace=frontend` : pass

Evidence

- Whole delta read from scratch, 26 commits, 89 files: backend/lib/settings (add, find and save of services, people and ticks; locks; name check), backend/routes/settings-routes.ts, the length rule (apply-length-change-rules.ts and its tests), the work email path (book-time.ts enqueue, find-booking-email-context.ts, find-booking-email-recipients.ts, send-booking-emails.ts, booking-email-job.ts, timeline line), the worker text change, the shared schemas, migration 0027 and its check, client setup and seed, and every frontend settings page, component and API client.
- Last commit 54dd2dd..9417aa7 read line by line: nine comment lines in seven code files lose their `(F-NNN)` tag and keep their sentence, plus the ledger; no behaviour change; no `F-NNN` left on any added code line.
- Security: every new route sits behind `requireOrganizationMiddleware`, the subscription and booking module checks, and writes behind `requirePermissionMiddleware({ organization: ["update"] })`; the business always comes from the session and every query filters on `organizationId` first; ids of another business, an unknown id or the wrong kind give 404 or 400 on the field (save-resource.ts, save-service.ts, save-service-resources.ts); the work email must be at the sending domain and a place cannot hold one (code and DB check); strict Zod schemas on every body; names and descriptions render as React text.

- Concurrency: people and places saves lock the business row for no key update (name_taken, last_person); the service save locks its booking_link row and reads the business hours row for no key update; addService settles slug races through the unique index with five tries; no lock cycle found with the hours saves or the booking path.
- Spec contracts matched: response shapes for GET/POST/PUT services, PUT resources, GET/POST/PUT people; refusal codes last_person, name_taken, no_sending_address; outsideHours empty when the length is unchanged; one test per rule listed under Testing, including the seven Simulate cases by name.
- Tests lens: no skipped, focused or placeholder tests added; route tests run against the real app and database.
- Standards: one export per file, Type suffixes, no em dashes in added code; comments beside their lines, except the step-number tags recorded as F-364.

Findings

- F-363 [P3] closed: the repair holds and introduced nothing new.
- F-364 [P3] open: nine comments added by this feature carry step-number tags such as (12d.4) , which coding-standards.md keeps out of code. Not blocking.
- No P0 or P1 finding is open or fixed.

Remaining risk

- The untracked blueprint/ai-voice-proposal.md is a path that differs from the target. The caller says it is the user's own unrelated note, so it was not reviewed and is not part of the work.
- Check was not required and was not run; no browser or hand check was done by this review, so the UI flows rest on the builder's recorded hand checks plus the route tests.
- The dashboard activity file (blueprint/.state/run.json) was not written, because this reviewer was told to write only review.md and findings.md.
- F-348 (a booking landing on a person while they are turned off) stays unverified, in the F-331 family.
- Observation only, not a finding: save-service.ts:32 and save-service-resources.ts:29 lock the booking_link row for update , which also holds back a booking insert of that service (its foreign key check takes a key share) for the length of the save. for no key update would serialize the saves just as well. The cost is milliseconds.
- Not reviewed: the 3072-line migration snapshot JSON (it was generated) and the build log in buildlogs/, which is outside this repo.